
Scenario

You are hired as the database engineer for **Wild West Grill**, a restaurant chain.
They want you to design and manage a MySQL database for their **Orders**, **Menu**, and **Staff** systems.

---

1. Database Setup

We'll create some base tables.

```
CREATE DATABASE wild_west_grill;
USE wild_west_grill;

-- Menu Table
CREATE TABLE Menu (
    menu_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    category VARCHAR(50),
    price DECIMAL(6,2)
);

-- Staff Table
CREATE TABLE Staff (
    staff_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    position VARCHAR(50),
    salary DECIMAL(8,2)
);

-- Orders Table
CREATE TABLE Orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    staff_id INT,
    menu_id INT,
    quantity INT,
    order_date DATE,
    FOREIGN KEY (staff_id) REFERENCES Staff(staff_id),
    FOREIGN KEY (menu_id) REFERENCES Menu(menu_id)
);
```

Insert Sample Data

```
INSERT INTO Menu (name, category, price) VALUES
('Cheeseburger', 'Main', 8.99),
('Fries', 'Side', 3.49),
('Coke', 'Drink', 1.99),
('Steak', 'Main', 18.50);
```

```
INSERT INTO Staff (name, position, salary) VALUES
('Alice', 'Waiter', 2000),
('Bob', 'Chef', 3000),
('Charlie', 'Waiter', 2100);

INSERT INTO Orders (staff_id, menu_id, quantity, order_date) VALUES
(1, 1, 2, '2025-08-01'),
(1, 2, 1, '2025-08-01'),
(2, 4, 3, '2025-08-02'),
(3, 1, 1, '2025-08-02'),
(3, 3, 2, '2025-08-03');
```

---

2. JOINS — Basics to Advanced

INNER JOIN — Get orders with staff and menu details

```
SELECT o.order_id, s.name AS staff_name, m.name AS item, o.quantity,
o.order_date
FROM Orders o
INNER JOIN Staff s ON o.staff_id = s.staff_id
INNER JOIN Menu m ON o.menu_id = m.menu_id;
```

LEFT JOIN — Show all staff, even if they have no orders

```
SELECT s.name, o.order_id
FROM Staff s
LEFT JOIN Orders o ON s.staff_id = o.staff_id;
```

RIGHT JOIN — Show all menu items, even if never ordered

```
SELECT m.name, o.order_id
FROM Orders o
RIGHT JOIN Menu m ON o.menu_id = m.menu_id;
```

FULL OUTER JOIN Simulation (MySQL doesn't have it directly)

```
SELECT s.name, o.order_id
FROM Staff s
LEFT JOIN Orders o ON s.staff_id = o.staff_id
UNION
SELECT s.name, o.order_id
FROM Staff s
RIGHT JOIN Orders o ON s.staff_id = o.staff_id;
```

SELF JOIN — Compare salaries between staff

```
SELECT a.name AS staff1, b.name AS staff2, a.salary - b.salary AS salary_diff
FROM Staff a
JOIN Staff b ON a.staff_id <> b.staff_id;
```

CROSS JOIN — All combinations of staff and menu

```
SELECT s.name, m.name
FROM Staff s
CROSS JOIN Menu m;
```

---

3. VIEWS — Basics to Advanced

Basic View — Daily Sales Summary

```
CREATE VIEW daily_sales AS
SELECT order_date, SUM(quantity * price) AS total_sales
FROM Orders o
JOIN Menu m ON o.menu_id = m.menu_id
GROUP BY order_date;
```

Usage:

```
SELECT * FROM daily_sales;
```

Updatable View — Only "Main" category menu

```
CREATE VIEW main_items AS
SELECT * FROM Menu WHERE category = 'Main';
-- Update via view
UPDATE main_items SET price = price + 1 WHERE name = 'Steak';
```

View with Join — Waiter Performance

```
CREATE VIEW waiter_sales AS
SELECT s.name AS waiter_name, SUM(o.quantity * m.price) AS sales
FROM Orders o
JOIN Staff s ON o.staff_id = s.staff_id
JOIN Menu m ON o.menu_id = m.menu_id
WHERE s.position = 'Waiter'
GROUP BY s.name;
```

---

4. PROCEDURES — Basics to Advanced

Basic Procedure — Get all orders by date

```
DELIMITER //
CREATE PROCEDURE GetOrdersByDate(IN p_date DATE)
BEGIN
    SELECT o.order_id, s.name, m.name AS item, o.quantity
    FROM Orders o
    JOIN Staff s ON o.staff_id = s.staff_id
    JOIN Menu m ON o.menu_id = m.menu_id
    WHERE o.order_date = p_date;
END //
DELIMITER ;
CALL GetOrdersByDate('2025-08-01');
```

Procedure with Output Parameter — Total Sales by Staff

```
DELIMITER //
CREATE PROCEDURE GetStaffSales(IN p_staff_id INT, OUT p_total DECIMAL(10,2))
BEGIN
    SELECT SUM(o.quantity * m.price)
    INTO p_total
    FROM Orders o
    JOIN Menu m ON o.menu_id = m.menu_id
    WHERE o.staff_id = p_staff_id;
END //
DELIMITER ;
CALL GetStaffSales(1, @total);
SELECT @total;
```

Procedure with Loop — Increase salary of all waiters

```
DELIMITER //
CREATE PROCEDURE IncreaseWaiterSalary(IN p_percent DECIMAL(5,2))
BEGIN
    UPDATE Staff
    SET salary = salary + (salary * p_percent / 100)
    WHERE position = 'Waiter';
END //
DELIMITER ;
CALL IncreaseWaiterSalary(10);
```

---

5. TRIGGERS — Basics to Advanced

Basic Trigger — Prevent negative salary

```
DELIMITER //
CREATE TRIGGER prevent_negative_salary
BEFORE UPDATE ON Staff          here 45000 means the
FOR EACH ROW                     SQL state code for
BEGIN                           a user-defined error
    IF NEW.salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be negative';
    END IF;
END //
DELIMITER ;
```

Audit Trigger — Track order changes

```
CREATE TABLE Order_Audit (
    audit_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT,
    old_quantity INT,
    new_quantity INT,
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //
CREATE TRIGGER track_order_update
BEFORE UPDATE ON Orders
FOR EACH ROW
BEGIN
    INSERT INTO Order_Audit (order_id, old_quantity, new_quantity)
    VALUES (OLD.order_id, OLD.quantity, NEW.quantity);
END //
DELIMITER ;
```

Trigger for Automatic Stock Update (Example with Stock table)

```
CREATE TABLE Stock (
    menu_id INT PRIMARY KEY,
    stock_qty INT,
    FOREIGN KEY (menu_id) REFERENCES Menu(menu_id)
);

INSERT INTO Stock VALUES (1, 20), (2, 50), (3, 100), (4, 10);

DELIMITER //
CREATE TRIGGER reduce_stock_after_order
AFTER INSERT ON Orders
FOR EACH ROW
BEGIN
```

```
    UPDATE Stock
    SET stock_qty = stock_qty - NEW.quantity
    WHERE menu_id = NEW.menu_id;
END //
DELIMITER ;
```

---

6. SCENARIO-BASED QUESTIONS & SOLUTIONS

Q1. Show all waiters and their total sales (even if 0)

```
SELECT s.name, COALESCE(SUM(o.quantity * m.price), 0) AS total_sales
FROM Staff s
LEFT JOIN Orders o ON s.staff_id = o.staff_id
LEFT JOIN Menu m ON o.menu_id = m.menu_id
WHERE s.position = 'Waiter'
GROUP BY s.name;
```

Q2. Find menu items never ordered

```
SELECT m.name
FROM Menu m
LEFT JOIN Orders o ON m.menu_id = o.menu_id
WHERE o.menu_id IS NULL;
```

Q3. List top 2 best-selling items

```
SELECT m.name, SUM(o.quantity) AS total_qty
FROM Orders o
JOIN Menu m ON o.menu_id = m.menu_id
GROUP BY m.name
ORDER BY total_qty DESC
LIMIT 2;
```

Q4. Procedure to delete old orders

```
DELIMITER //
CREATE PROCEDURE DeleteOldOrders(IN p_date DATE)
BEGIN
    DELETE FROM Orders WHERE order_date < p_date;
END //
DELIMITER ;
CALL DeleteOldOrders('2025-08-02');
```

Q5. Trigger to auto-insert today's date if order_date is NULL

```
DELIMITER //
CREATE TRIGGER set_order_date
BEFORE INSERT ON Orders
FOR EACH ROW
BEGIN
    IF NEW.order_date IS NULL THEN
        SET NEW.order_date = CURDATE();
    END IF;
END //
DELIMITER ;
```

Wild West Grill – Passage-Based MySQL Problem Set

Passage 1 – The Opening Day (Basic Joins)

Wild West Grill has just opened.
The owner wants a list of all orders placed so far, showing **order ID**, **staff name**, **menu item name**, and **quantity**.
The data is stored in three tables: `Orders`, `Staff`, and `Menu`.

Question:
Write a query to get the required information.

Solution:

```
SELECT o.order_id, s.name AS staff_name, m.name AS menu_item, o.quantity
FROM Orders o
JOIN Staff s ON o.staff_id = s.staff_id
JOIN Menu m ON o.menu_id = m.menu_id;
```

Passage 2 – The Missing Waiter (LEFT JOIN)

The manager wants to see **all staff members**, including those who have **not taken any orders yet**.

Question:
Write a query to list each staff member's name and their order IDs (show `NULL` if none).

Solution:

```
SELECT s.name, o.order_id
FROM Staff s
LEFT JOIN Orders o ON s.staff_id = o.staff_id;
```

Passage 3 – Forgotten Dishes (RIGHT JOIN)

Some menu items have never been ordered. The chef wants to find them.

Question:
Write a query to show menu items and their order IDs, including items with no orders.

Solution:

```
SELECT m.name, o.order_id
FROM Orders o
RIGHT JOIN Menu m ON o.menu_id = m.menu_id;
```

Passage 4 – Daily Sales Report (Basic View)

The manager is tired of writing the sales query every day. He asks you to create a **view** that shows **date** and **total sales**.

Question:
Create a view `daily_sales` to display the order date and total sales.

Solution:

```
CREATE VIEW daily_sales AS
SELECT o.order_date, SUM(o.quantity * m.price) AS total_sales
FROM Orders o
JOIN Menu m ON o.menu_id = m.menu_id
GROUP BY o.order_date;
```

Passage 5 – Custom Report (Procedure)

The manager wants to quickly get all orders for a specific date without writing the full query.

Question:
Write a stored procedure `GetOrdersByDate` that takes a date as input and returns all order details.

Solution:

```
DELIMITER //
CREATE PROCEDURE GetOrdersByDate(IN p_date DATE)
BEGIN
    SELECT o.order_id, s.name, m.name AS menu_item, o.quantity
    FROM Orders o
    JOIN Staff s ON o.staff_id = s.staff_id
    JOIN Menu m ON o.menu_id = m.menu_id
    WHERE o.order_date = p_date;
END //
DELIMITER ;
```

Passage 6 – Staff Performance Bonus (Procedure with Output)

The owner decides to give bonuses to staff with sales over $200.
You need a procedure to calculate **total sales** for a given staff member.

Question:
Create a procedure `GetStaffSales` with staff ID as input and total sales as output.

Solution:

```
DELIMITER //
CREATE PROCEDURE GetStaffSales(IN p_staff_id INT, OUT p_total DECIMAL(10,2))
BEGIN
    SELECT SUM(o.quantity * m.price)
    INTO p_total
    FROM Orders o
    JOIN Menu m ON o.menu_id = m.menu_id
    WHERE o.staff_id = p_staff_id;
END //
DELIMITER ;
```

Passage 7 – Salary Protection (Trigger)

An HR intern accidentally tried to set a waiter's salary to `-100`. The system must **block negative salaries** automatically.

Question:
Create a trigger that prevents salary updates if the value is negative.

Solution:

```
DELIMITER //
CREATE TRIGGER prevent_negative_salary
```

```
BEFORE UPDATE ON Staff
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be negative';
    END IF;
END //
DELIMITER ;
```

Passage 8 – Stock Management (Trigger with AFTER INSERT)

The restaurant keeps a stock count for each menu item. When a new order is inserted, stock must be reduced automatically.

Question:
Write a trigger to reduce stock after an order is placed.

Solution:

```
DELIMITER //
CREATE TRIGGER reduce_stock_after_order
AFTER INSERT ON Orders
FOR EACH ROW
BEGIN
    UPDATE Stock
    SET stock_qty = stock_qty - NEW.quantity
    WHERE menu_id = NEW.menu_id;
END //
DELIMITER ;
```

Passage 9 – Waiter Sales Report (Advanced View)

The manager wants a **view** showing each waiter's total sales.

Question:
Create a view `waiter_sales` with waiter name and their total sales.

Solution:

```
CREATE VIEW waiter_sales AS
SELECT s.name AS waiter_name, SUM(o.quantity * m.price) AS sales
FROM Orders o
JOIN Staff s ON o.staff_id = s.staff_id
JOIN Menu m ON o.menu_id = m.menu_id
WHERE s.position = 'Waiter'
GROUP BY s.name;
```

Passage 10 – Menu Without Orders (Scenario Query)

Some items have never been ordered, and the chef wants to consider removing them from the menu.

Question:
Write a query to list all such items.

Solution:

```
SELECT m.name
FROM Menu m
LEFT JOIN Orders o ON m.menu_id = o.menu_id
WHERE o.menu_id IS NULL;
```

Passage 11 – Top-Selling Items (Advanced Query with LIMIT)

The marketing team wants the **top 3 best-selling dishes** for a new poster.

Question:
Write a query to find the top 3 menu items based on quantity sold.

Solution:

```
SELECT m.name, SUM(o.quantity) AS total_qty
FROM Orders o
JOIN Menu m ON o.menu_id = m.menu_id
GROUP BY m.name
ORDER BY total_qty DESC
LIMIT 3;
```
